

Design a Rate Limiter - High-Level Architecture

1. Core Idea

The basic idea behind rate limiting algorithms:

- Maintain a **counter** for each entity (user, IP address, etc.)
- Track number of requests within a time window

None

If counter > limit → Reject request

Else → Allow request

2. Where to Store Counters?

Database (Not Preferred)

- Disk access is **slow**
- Not suitable for high-performance rate limiting

In-Memory Cache (Preferred)

- Fast access (RAM-based)
- Supports **time-based expiration**

Example: Redis

Redis is commonly used because it provides:

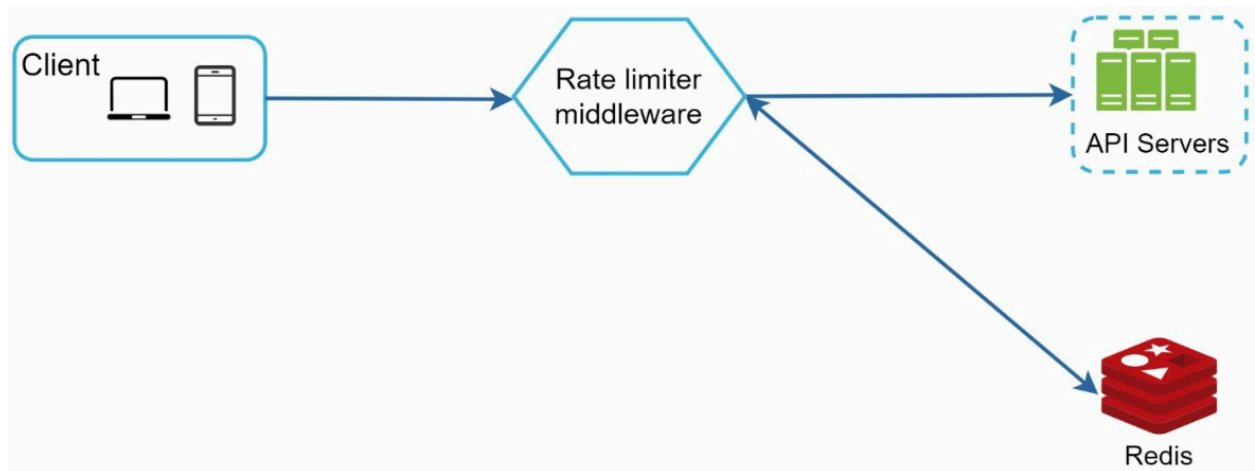
None

`INCR` → Increment counter by 1

`EXPIRE` → Set timeout for counter

- Counter automatically deleted after expiry
- Efficient for time-window based rate limiting

3. High-Level Architecture Flow



Step-by-Step Flow

1. **Client → Rate Limiter Middleware**
 - Client sends request
2. **Middleware → Cache (Redis)**
 - Fetch counter for user/IP
3. **Check Limit**
 - If limit reached → reject request
 - If not → proceed
4. **Forward Request**
 - Send request to API servers
5. **Update Counter**
 - Increment counter using **INCR**
 - Apply expiration using **EXPIRE**

4. Key Takeaway

Fast, scalable rate limiting requires:

- **In-memory counters**
- **Efficient increment operations**
- **Automatic expiration mechanism**